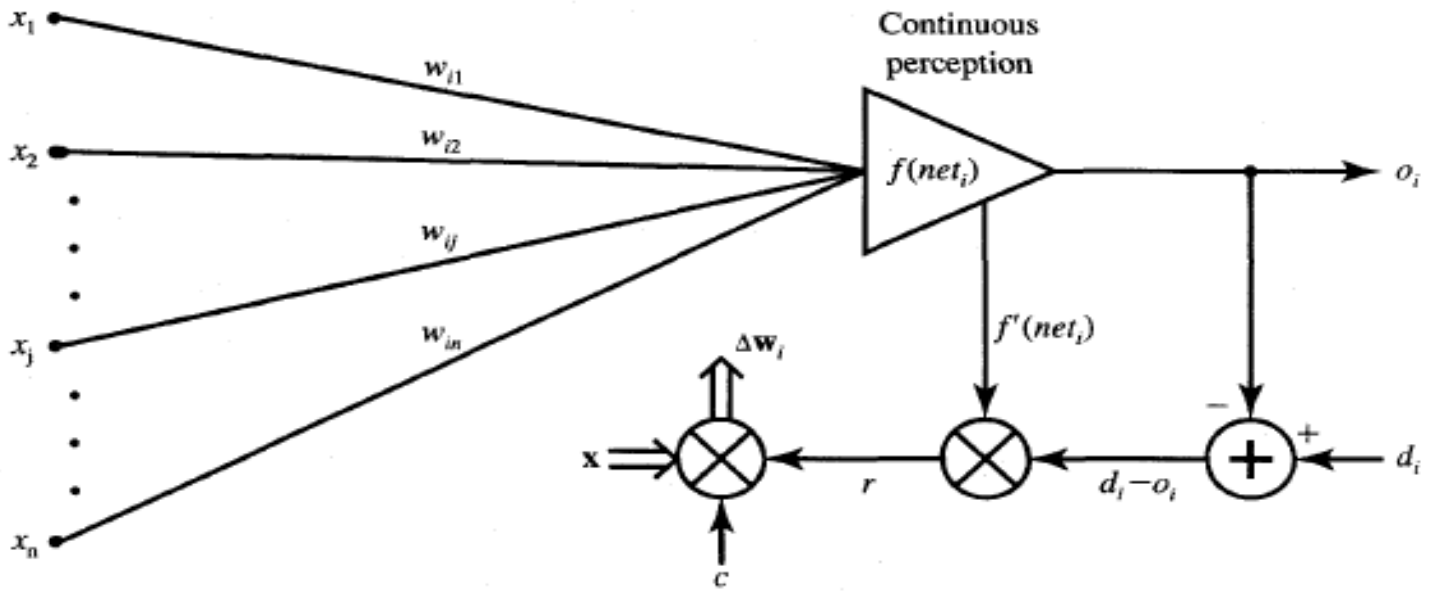
Delta Learning Rule

- Only valid for continuous activation function
- Used in supervised training mode
- Learning signal for this rule is called delta
- The aim of the delta rule is to minimize the error over all training patterns



Mathematical Definition

For the jth neuron of a neural network with activation function $g(x)$, the delta rule for updating the neuron's ith weight, w_{ji} is given by:

$$\Delta w_{ji} = \alpha(t_j - y_j)g'(h_j)x_i$$

where
α is the learning rate,
g' is the derivative of the activation function g,
t_j is the target output,
h_j is the weighted sum of the neuron's inputs obtained as $\sum x_i w_{ji}$,
y_j is the predicted output,
x_i is the ith input.

For a neuron with a linear activation function, the derivative of the activation function is constant and so the delta rule in this case can be simplified as:

$$\Delta w_{ji} = \alpha(t_j - y_j)x_i$$

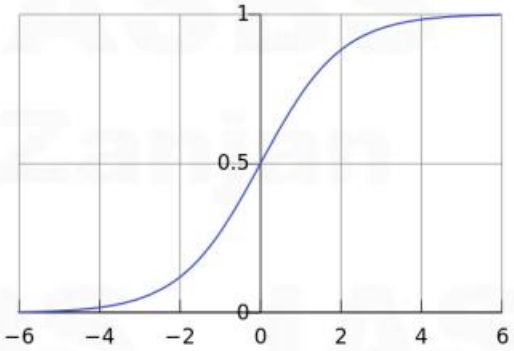
Delta Learning Rule **vs.** **Perceptron Learning Rule**

$$\Delta w_{ji} = \eta\ (\text{targ}_j - \text{out}_j)\ f'(net_j)\ x_i$$

$$\Delta w_{ij} = \eta\ (\text{targ}_j - \text{out}_j)\ in_i$$

If we used Sigmoid / Logistic Activation Function

$$\Delta w_{ji} = \eta\ (\text{targ}_j - \text{out}_j)\ f'(net_j)\ x_i$$



We noted earlier that the Sigmoid is a smooth threshold function:

$$f(x) = sigmoid(x) = \frac{1}{1+e^{-x}}$$

We can use the chain rule: $f(x) = g(h(x))$

$$g(h) = h^{-1} \qquad h(x) = 1 + e^{-x} \qquad \frac{\partial g(h)}{\partial h} = -\frac{1}{h^2} \qquad \frac{\partial h(x)}{\partial x} = -e^{-x}$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g}\frac{\partial g}{\partial h}\frac{\partial h}{\partial x} = -\frac{1}{(1+e^{-x})^2}\cdot(-e^{-x}) = \underbrace{\left(\frac{1}{1+e^{-x}}\right)}_{f(x)}\underbrace{\left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)}_{1-f(x)}$$

$$f'(x) = f(x).(1 - f(x))$$

If we used Sigmoid / Logistic Activation Function

If we use the Sigmoid activation function, due to the properties of the Sigmoid derivative, the general weight update equation simplifies so that it only contains neuron activations and no derivatives:

Delta rule: $$\Delta w_{ji} = \eta\ (\text{targ}_j - \text{out}_j)\ f'(net_j)\ x_i$$

$$f'(net) = f(net).(1 - f(net))$$

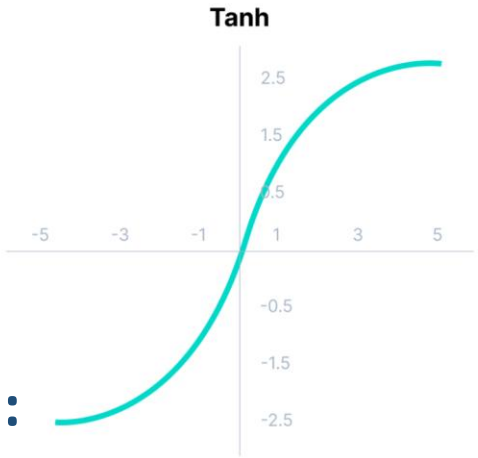
Generalized Delta Rule

$$\Delta w_{ji} = \eta\ (\text{targ}_j - \text{out}_j)\ out_j(1 - out_j)\ x_i$$

Remember, If we use the Sigmoid activation function

If we used Tanh Function (Hyperbolic Tangent)

$$f(x) = \frac{\left(e^{x} - e^{-x}\right)}{\left(e^{x} + e^{-x}\right)} = \frac{2}{1+e^{-2x}} - 1$$



Derivative of Tanh Function (Hyperbolic Tangent):

$$\frac{d}{dz}\left(\frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}\right) = \frac{e^{z} + e^{-z}}{\left(e^{z} + e^{-z}\right)^{2}} d\left(e^{z} - e^{-z}\right) - \frac{e^{z} - e^{-z}}{\left(e^{z} + e^{-z}\right)^{2}} d\left(e^{z} + e^{-z}\right)$$

$$= \frac{\left(e^{z} + e^{-z}\right)\left(e^{z} + e^{-z}\right)}{\left(e^{z} + e^{-z}\right)^{2}} - \frac{\left(e^{z} - e^{-z}\right)\left(e^{z} - e^{-z}\right)}{\left(e^{z} + e^{-z}\right)^{2}}$$

$$= \frac{\left(e^{z} + e^{-z}\right)^{2} - \left(e^{z} - e^{-z}\right)^{2}}{\left(e^{z} + e^{-z}\right)^{2}}$$

$$= 1 - \left(\frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}\right)^{2}$$

$$= 1 - tanh(z)^{2}$$

How do we choose the initial weights to start the training?

- we generally start off all the weights with small random values to avoid saturation of activation function
- A uniform or normal random distribution can be used
- we need to check the independency of network performance by starting with different initial random weights

How do we choose an appropriate learning rate η ?

- If η is too small → take too long to get anywhere near the minimum
- If η is too large → weights will oscillate, or even diverge
- No necessity to keep the learning rate fixed throughout the learning

Solve the following classification problem with the Delta learning rule. Apply each input vector in order, make only one epoch.

$$\left\{P_1 = \begin{bmatrix} 1 \\ -2 \\ 0 \\ -1 \end{bmatrix}, t_1 = -1\right\}\left\{P_2 = \begin{bmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{bmatrix}, t_2 = -1\right\}\left\{P_3 = \begin{bmatrix} -1 \\ 1 \\ 0.5 \\ -1 \end{bmatrix}, t_3 = 1\right\}$$

Initial weight

$$b(0) = 1,\ w(0) = [1 \quad -1 \quad 0 \quad 0.5],\ \alpha = 0.1$$

$$\left\{P_1 = \begin{bmatrix}1\\-2\\0\\-1\end{bmatrix}, t_1 = -1\right\}\left\{P_2 = \begin{bmatrix}0\\1.5\\-0.5\\-1\end{bmatrix}, t_2 = -1\right\}\left\{P_3 = \begin{bmatrix}-1\\1\\0.5\\-1\end{bmatrix}, t_3 = 1\right\}$$

For the first epoch:

$$w(0) = [1 \quad -1 \quad 0 \quad 0.5],\ \alpha = 0.1$$

For the first input vector:

$$n = w(0)P_1$$

$$n = [1 \quad -1 \quad 0 \quad 0.5]\begin{bmatrix}1\\-2\\0\\-1\end{bmatrix} = 2.5$$

$$f(n) = a = tanh(n) = tanh(2.5) = 0.9866$$

$$\grave{f}(n) = 1 - tanh^2 n = 0.0266$$

$$e = t_1 - a = -1.9866$$

$$w(1) = w(0) + \alpha e \grave{f}(n) P_1^T$$

$$w(1) = [1 \quad -1 \quad 0 \quad 0.5] + (0.1)(-1.9866)(0.0266)[1 \quad -2 \quad 0 \quad -1]$$

$$w(1) = [0.9947 \quad -0.9894 \quad 0 \quad 0.5053]$$

Remember

$$\Delta w_{ji} = \eta\ (\text{targ}_j - \text{out}_j)\ f'(net_j)\ x_i$$

$$\left\{P_1 = \begin{bmatrix}1\\-2\\0\\-1\end{bmatrix}, t_1 = -1\right\}\left\{P_2 = \begin{bmatrix}0\\1.5\\-0.5\\-1\end{bmatrix}, t_2 = -1\right\}\left\{P_3 = \begin{bmatrix}-1\\1\\0.5\\-1\end{bmatrix}, t_3 = 1\right\}$$

For the first epoch:

$$, \; w(0) = [1 \quad -1 \quad 0 \quad 0.5], \alpha = 0.1$$

For the second input vector:

$$n = w(1)P_2$$

$$n = [0.9947 \quad -0.9894 \quad 0 \quad 0.5053]\begin{bmatrix}0\\1.5\\-0.5\\-1\end{bmatrix} + 0 = -1.9894$$

$$f(n) = a = tanh(n) = -0.9633$$

$$\grave{f}(n) = 1 - tanh^2 n = 0.0721$$

$$e = t_2 - a = -0.0367$$

$$w(2) = w(1) + \alpha e \grave{f}(n) P_2^T$$

$$w(2) = [0.9947 \quad -0.9894 \quad 0 \quad 0.5053] + (0.1)(-0.0367)(0.0721)[0 \quad 1.5 \quad -0.5 \quad -1]$$

$$w(2) = [0.9947 \quad -0.9898 \quad 0.0001 \quad 0.5055]$$

Remember

$$\Delta w_{ji} = \eta \, (\text{targ}_j - \text{out}_j) \; f'(net_j) \; x_i$$

$$\left\{P_1 = \begin{bmatrix}1\\-2\\0\\-1\end{bmatrix}, t_1 = -1\right\}\left\{P_2 = \begin{bmatrix}0\\1.5\\-0.5\\-1\end{bmatrix}, t_2 = -1\right\}\left\{P_3 = \begin{bmatrix}-1\\1\\0.5\\-1\end{bmatrix}, t_3 = 1\right\}$$

For the first epoch:

$$w(0) = [1 \quad -1 \quad 0 \quad 0.5],\ \alpha = 0.1$$

For the third input vector:

$$n = w(2)P_3$$

$$n = [0.9947 \quad -0.9898 \quad 0.0001 \quad 0.5055]\begin{bmatrix}-1\\1\\0.5\\-1\end{bmatrix} = -2.49$$

$$f(n) = a = tanh(n) = -0.9863$$

$$\grave{f}(n) = 1 - tanh^2 n = 0.0721$$

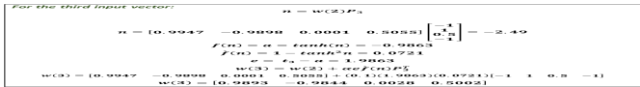
$$e = t_3 - a = 1.9863$$

$$w(3) = w(2) + \alpha e \grave{f}(n) P_3^T$$

$$w(3) = [0.9947 \quad -0.9898 \quad 0.0001 \quad 0.5055] + (0.1)(1.9863)(0.0721)[-1 \quad 1 \quad 0.5 \quad -1]$$

$$w(3) = [0.9893 \quad -0.9844 \quad 0.0028 \quad 0.5002]$$

Remember



Python code for Delta Learning Rule

```
[1] import numpy as np
```

```
x = np.array([[1, -2, 0, -1],
              [0, 1.5, -0.5, -1],
              [-1, 1, 0.5, -1]])

d = np.array([[-1],
              [-1],
              [1]])

w = np.array([1, -1, 0, 0.5])

c = 0.1

epochno = 25
```

```
[3] # calculateing output
    def calculate_output(weights, instance):
        sum = np.dot(instance, w)
        return np.tanh(sum)
```

```
for j in range(epochno):
  print(["epoch"+str(j)])
  for i in range(3):
      net = np.dot(x[i, :], w)
      fnet = np.tanh(net)
      fnetd = 1 - np.tanh(net) ** 2
      e = d[i, 0] - fnet
      deltaw = c * e * fnetd * x[i, :]

      w = w + deltaw
  print(w)
  print('prediction for [1, -2, 0, -1]: ' + str(calculate_output(w, np.array([[1, -2, 0, -1]]))))
  print('prediction for [0, 1.5, -0.5, -1]: ' + str(calculate_output(w, np.array([[0, 1.5, -0.5, -1]]))))
  print('prediction for [-1, 1, 0.5, -1]: ' + str(calculate_output(w, np.array([[-1, 1, 0.5, -1]]))))
```

Python code for Delta Learning Rule

```
['epoch0']
[ 0.98933008 -0.98444445  0.00282594  0.5001606 ]
prediction for [1, -2, 0, -1]: [0.98545154]
prediction for [0, 1.5, -0.5, -1]: [-0.96245755]
prediction for [-1, 1, 0.5, -1]: [-0.98586343]
['epoch1']
[ 0.97790078 -0.96771393  0.00581768  0.50049055]
prediction for [1, -2, 0, -1]: [0.9840854]
prediction for [0, 1.5, -0.5, -1]: [-0.96070417]
prediction for [-1, 1, 0.5, -1]: [-0.9850159]
['epoch2']
[ 0.96559375 -0.94961874  0.00899765  0.50103124]
prediction for [1, -2, 0, -1]: [0.98245983]
prediction for [0, 1.5, -0.5, -1]: [-0.95873002]
prediction for [-1, 1, 0.5, -1]: [-0.98405084]
['epoch3']
[ 0.95226063 -0.92991995  0.0123934   0.50183713]
prediction for [1, -2, 0, -1]: [0.98049685]
prediction for [0, 1.5, -0.5, -1]: [-0.95648621]
prediction for [-1, 1, 0.5, -1]: [-0.98294072]
['epoch22']
[-0.40844816  0.2942707   0.6579735   0.51662592]
prediction for [1, -2, 0, -1]: [-0.90757856]
prediction for [0, 1.5, -0.5, -1]: [-0.38354254]
prediction for [-1, 1, 0.5, -1]: [0.47389359]
['epoch23']
[-0.46322136  0.27125367  0.71101861  0.51805794]
prediction for [1, -2, 0, -1]: [-0.90935529]
prediction for [0, 1.5, -0.5, -1]: [-0.43551837]
prediction for [-1, 1, 0.5, -1]: [0.51677263]
['epoch24']
[-0.51003568  0.25052735  0.75667784  0.52045422]
prediction for [1, -2, 0, -1]: [-0.91068855]
prediction for [0, 1.5, -0.5, -1]: [-0.48001373]
prediction for [-1, 1, 0.5, -1]: [0.55004633]
['epoch25']
[-0.55082164  0.23236373  0.79646668  0.52303216]
prediction for [1, -2, 0, -1]: [-0.91188166]
prediction for [0, 1.5, -0.5, -1]: [-0.51735398]
prediction for [-1, 1, 0.5, -1]: [0.57728867]
```

Linearly Separable & not Linearly Separable problems

- Functions which can be separated with linear line or lines are called Linearly Separable
- Only linearly separable functions can be represented by a perceptron
- XOR cannot be represented by a perceptron

  XOR is not linearly separable
- What can we do instead?
  - Convert to logic gates that can be represented by perceptron
  - Chain together the gates
- Make sure you understand the following
  - check it using truth tables

$$x_1 \text{ XOR } x_2 = (x_1 \text{ OR } x_2) \text{ AND } (\text{NOT}(x_1 \text{ AND } x_2))$$